

Functional Programming

Patrick Bahr

Mutual Recursion
& Mutable Data

First 4 Weeks of FP

We have now covered the main pillars of functional programming:

First 4 Weeks of FP

We have now covered the main pillars of functional programming:

- (Recursive) Algebraic data types
- Pattern matching
- Higher-order and recursive functions
- Polymorphic types and functions

This week

I. Mutual Recursion

II. Mutable data

- ▶ Mutable variables
- ▶ Mutable data structures

III. Arrays

Recap

Recursion

Recursive Function Definitions

```
let rec fact (x : int) : int =  
  match x with  
  | 0 -> 1  
  | x -> x * fact (x - 1)
```


Recursive Function Definitions

```
let rec fact (x : int) : int =  
  match x with  
  | 0 -> 1  
  | x -> x * fact (x - 1)
```

Recursive Types

```
type Tree =  
  | Leaf  
  | Node of Tree * int * Tree
```


Recursive Types

```
type Tree =  
  | Leaf  
  | Node of Tree * int * Tree
```

```
let rec sum (t : Tree) : int =  
  match t with  
  | Leaf          -> 0  
  | Node (l,n,r) -> sum l + sum r + n
```

Part I

Mutual Recursion

Mutual recursion

Recursive function

```
let rec fact (x : int) : int =  
  match x with  
  | 0 -> 1  
  | x -> x * fact (x - 1)
```


Mutual recursion

Recursive function

```
let rec fact (x : int) : int =  
  match x with  
  | 0 -> 1  
  | x -> x * fact (x - 1)
```

Mutual recursive function:

```
let rec isEven n = if n = 0 then true  
                  else isOdd (n - 1)  
  
and isOdd n = if n = 0 then false  
              else isEven (n - 1)
```

Mutual recursion

Recursive function

```
let rec fact (x : int) : int =  
  match x with  
  | 0 -> 1  
  | x -> x * fact (x - 1)
```

Mutual recursive function:

```
let rec isEven n = if n = 0 then true  
                  else isOdd (n - 1)  
  
and isOdd n = if n = 0 then false  
              else isEven (n - 1)
```

Mutual recursion

Recursive function

```
let rec fact (x : int) : int =
  match x with
  | 0 -> 1
  | x -> x * fact (x - 1)
```

Mutual recursive function:

```
let rec isEven n = if n = 0 then true
                  else isOdd (n - 1)

and isOdd n = if n = 0 then false
              else isEven (n - 1)
```


Mutual recursion

Recursive function

```
let rec fact (x : int) : int =
  match x with
  | 0 -> 1
  | x -> x * fact (x - 1)
```

Mutual recursive function:

```
let rec isEven n = if n = 0 then true
                  else isOdd (n - 1)
```

```
and isOdd n = if n = 0 then false
               else isEven (n - 1)
```

Mutual recursion

Recall the **recursive type** of trees:

```
type Tree = Leaf  
          | Node of Tree * int * Tree
```


Mutual recursion

Recall the **recursive type** of trees:

```
type Tree = Leaf
           | Node of Tree * int * Tree
```

Mutual recursive type declaration:

```
type RoseTree = Leaf
               | Node of int * Children
and Children   = RoseTree list
```

Two types that are defined in terms of each other.

Mutual recursion

Recall the **recursive type** of trees:

```
type Tree = Leaf
           | Node of Tree * int * Tree
```

Mutual recursive type declaration:

```
type RoseTree = Leaf
               | Node of int * Children
and Children = RoseTree list
```

Two types that are defined in terms of each other.

Mutual recursion

Recall the **recursive type** of trees:

```
type Tree = Leaf
           | Node of Tree * int * Tree
```

Mutual recursive type declaration:

```
type RoseTree = Leaf
              | Node of int * Children
and Children = RoseTree list
```

Two types that are defined in terms of each other.

Mutual recursion

Recall the **recursive type** of trees:

```
type Tree = Leaf
           | Node of Tree * int * Tree
```

Mutual recursive type declaration:

```
type RoseTree = Leaf
           | Node of int * Children
and Children = RoseTree list
```

Two types that are defined in terms of each other.

Mutual recursion

To traverse mutual recursive types we need **mutually recursive functions**, i.e. functions that call each other:

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n, ts) -> n + sumChildren ts  
  
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```

Mutual recursion

To traverse mutual recursive types we need **mutually recursive functions**, i.e. functions that call each other:

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n, ts) -> n + sumChildren ts
```

```
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```


Mutual recursion

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n,ts) -> n + sumChildren ts  
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```

Mutual recursion

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n,ts) -> n + sumChildren ts  
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```

```
sumRose (Node (4, [Leaf;Leaf]))
```


Mutual recursion

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n,ts) -> n + sumChildren ts  
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```

```
sumRose (Node (4, [Leaf;Leaf]))  
~> 4 + sumChildren [Leaf;Leaf]
```

Mutual recursion

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n,ts) -> n + sumChildren ts  
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```

sumRose (Node (4, [Leaf;Leaf]))

~> 4 + sumChildren [Leaf;Leaf]

~> 4 + (sumRose Leaf + sumChildren [Leaf])

Mutual recursion

```
let rec sumRose (t : RoseTree) : int =
  match t with
  | Leaf -> 0
  | Node (n,ts) -> n + sumChildren ts
and sumChildren (ch : Children) : int =
  match ch with
  | [] -> 0
  | t :: ts -> sumRose t + sumChildren ts
```

```
sumRose (Node (4, [Leaf;Leaf]))
~> 4 + sumChildren [Leaf;Leaf]
~> 4 + (sumRose Leaf + sumChildren [Leaf])
~> 4 + (0 + (sumRose Leaf + sumChildren []))
```


Mutual recursion

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n,ts) -> n + sumChildren ts  
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```

```
sumRose (Node (4, [Leaf;Leaf]))  
~> 4 + sumChildren [Leaf;Leaf]  
~> 4 + (sumRose Leaf + sumChildren [Leaf])  
~> 4 + (0 + (sumRose Leaf + sumChildren []))  
~> 4 + (0 + (0 + 0))
```

Mutual recursion

```
let rec sumRose (t : RoseTree) : int =  
  match t with  
  | Leaf -> 0  
  | Node (n,ts) -> n + sumChildren ts  
and sumChildren (ch : Children) : int =  
  match ch with  
  | [] -> 0  
  | t :: ts -> sumRose t + sumChildren ts
```

```
sumRose (Node (4, [Leaf;Leaf]))  
~> 4 + sumChildren [Leaf;Leaf]  
~> 4 + (sumRose Leaf + sumChildren [Leaf])  
~> 4 + (0 + (sumRose Leaf + sumChildren []))  
~> 4 + (0 + (0 + 0)) ~> 4
```

Two Recursive Types

Boolean expressions may contain arithmetic expressions

```
type aexpr = Num of int  
           | Add of aexpr * aexpr
```

```
type bexpr = Eq of aexpr * aexpr  
           | Not of bexpr
```


... Extended to Mutual Recursion

Boolean expressions may contain arithmetic expressions
and vice versa.

```
type aexpr = Num of int
           | Add of aexpr * aexpr
           | If  of bexpr * aexpr * aexpr

and bexpr = Eq  of aexpr * aexpr
           | Not of bexpr
```


... Extended to Mutual Recursion

Boolean expressions may contain arithmetic expressions
and vice versa.

```
type aexpr = Num of int
           | Add of aexpr * aexpr
           | If of bexpr * aexpr * aexpr
           | ...

and bexpr = Eq of aexpr * aexpr
           | Not of bexpr
```

... Extended to Mutual Recursion

Boolean expressions may contain arithmetic expressions
and vice versa.

```
type aexpr = Num of int
           | Add of aexpr * aexpr
           | If of bexpr * aexpr * aexpr
           | ...

and bexpr = Eq of aexpr * aexpr
           | Not of bexpr
```



... Extended to Mutual Recursion

Boolean expressions may contain arithmetic expressions
and vice versa.

type **aexpr** = Num of int
 | Add of **aexpr** * **aexpr**
 → | If of **bexpr** * **aexpr** * **aexpr**

and **bexpr** = Eq of **aexpr** * **aexpr**
 | Not of **bexpr**

Mutual Recursive Evaluator



Mutual Recursive Evaluator

```
type aexpr = Num of int
           | Add of aexpr * aexpr
           | If  of bexpr * aexpr * aexpr

and bexpr = Eq  of aexpr * aexpr
           | Not of bexpr
```

```
let rec aeval (e : aexpr) : int =
  match e with
  | Num n          -> n
  | Add (x, y)     -> aeval x + aeval y
  | If (b, x, y)   -> if beval b then aeval x else aeval y
```

```
and beval (e : bexpr) : bool =
  match e with
  | Eq (x, y) -> aeval x = aeval y
  | Not x     -> not (beval x)
```

Mutual Recursion in the Wild

For example, the JSON data format:

```
type JsonValue =  
  | JString of string  
  | JNumber of float  
  | JObject of JsonObject  
  | JArray  of JsonArray
```

```
and JsonObject = Map<string, JsonValue>
```

```
and JsonArray = JsonValue list
```


Mutual Recursion in the Wild

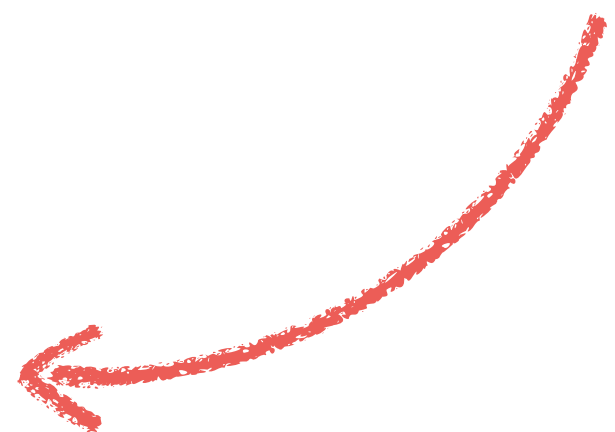
For example, the JSON data format:

```
type JsonValue =  
  | JString of string  
  | JNumber of float  
  | JObject of JsonObject  
  | JArray  of JsonArray
```

```
and JsonObject = Map<string, JsonValue>
```

```
and JsonArray = JsonValue list
```

More on maps
next week



Mutual Recursion in the Wild

For example, the DOM tree of a browser:

```
type Node =  
  | Text      of string  
  | Element  of ElementNode  
  | ...  
  
and ElementNode = {  
  TagName    : string  
  Children   : Node list  
  ...  
}
```

Questions?

Part II

Mutable Data

Imperative features of F#

F# is a functional programming language, but it also has some **imperative** features:

- References
- Sequencing
- Mutable data
- Arrays
- Loops

Imperative features of F#

F# is a functional programming language, but it also has some **imperative** features:

- References
- Sequencing
- Mutable data
- Arrays
- Loops

We will cover some of these imperative features today.

More will follow in later lectures.

Variables are immutable (by default)

```
> let x = 15;;  
val x : int = 15  
  
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 15
```

Values assigned to a variable never change.

Calling the function `f` does not change the value of `x`
no matter how `f` is defined!

Mutable Variables

Mutable variables are defined using the `mutable` keyword:

Immutable variable:

```
> let x = 10;;  
val x : int = 10
```

Mutable variable:

```
> let mutable x = 10;;  
val mutable x : int = 10
```

- They behave like variables in imperative languages
- They cannot be aliased
- They are mutated using the `<-` operator

Mutable Variables

Mutable variables are defined using the `mutable` keyword:

Immutable variable:

```
> let x = 10;;  
val x : int = 10
```

- They behave like variables in imperative languages
- They cannot be aliased
- They are mutated using the `<-` operator

Mutable variable:

```
> let mutable x = 10;;  
val mutable x : int = 10  
  
> x;;  
val it : int = 10
```

Mutable Variables

Mutable variables are defined using the `mutable` keyword:

Immutable variable:

```
> let x = 10;;  
val x : int = 10
```

- They behave like variables in imperative languages
- They cannot be aliased
- They are mutated using the `<-` operator

Mutable variable:

```
> let mutable x = 10;;  
val mutable x : int = 10  
  
> x;;  
val it : int = 10  
  
> x <- 15;;  
val it : unit = ()
```

Mutable Variables

Mutable variables are defined using the `mutable` keyword:

Immutable variable:

```
> let x = 10;;  
val x : int = 10
```

- They behave like variables in imperative languages
- They cannot be aliased
- They are mutated using the `<-` operator

Mutable variable:

```
> let mutable x = 10;;  
val mutable x : int = 10
```

```
> x;;  
val it : int = 10
```

```
> x <- 15;;  
val it : unit = ()
```

```
> x;;  
val it : int = 15
```


Immutable vs Mutable Variables

```
> let x = 15;;  
val x : int = 15  
  
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 15
```

← This is the example
with an **immutable**
variable from earlier.

Let's compare this to
an example with a
mutable variable.

Immutable vs Mutable Variables

```
> let x = 15;;  
val x : int = 15  
  
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 15
```

```
> let mutable x = 15;;  
val mutable x : int = 15  
  
> let f y =  
    x <- y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 3
```

Immutable vs Mutable Variables

```
> let x = 15;;  
val x : int = 15  
  
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 15
```

```
> let mutable x = 15;;  
val mutable x : int = 15  
  
> let f y =  
    x <- y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 3
```


Immutable vs Mutable Variables

```
> let x = 15;;  
val x : int = 15  
  
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 15
```

```
> let mutable x = 15;;  
val mutable x : int = 15  
  
> let f y =  
    x <- y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 3
```


Immutable vs Mutable Variables

```
> let x = 15;;  
val x : int = 15  
  
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 15
```

```
> let mutable x = 15;;  
val mutable x : int = 15  
  
> let f y =  
    x <- y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 3
```

Reference Cells

Here we create a **reference cell** containing the value 5:

```
> let x = ref 5;;  
val x : int ref = {contents = 5}
```

A value of type τ ref is a reference cell containing a value of type τ .

We read (aka dereference) the value of a reference cell with **!**

```
> !x;;  
val it : int = 5
```

We update a reference cell with **:=**

```
> x := 6;;  
val it : unit = ()  
> !x;;  
val it : int = 6
```

Values vs Reference Cells

```
> let x = 15;;  
val x : int = 15  
  
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int  
  
> f 2;;  
val it : int = 9  
  
> x;;  
val it : int = 15
```

← This is the example
with an **immutable**
variable from earlier.

Let's compare this to
an example with a
reference cell.

Values vs Reference Cells

```
> let x = 15;;  
val x : int = 15
```

```
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int
```

```
> f 2;;  
val it : int = 9
```

```
> x;;  
val it : int = 15
```

```
> let x = ref 15;;  
val x : int ref = { contents = 15 }
```

```
> let f y =  
    x := y + 1  
    !x * !x;;  
val f : y:int -> int
```

```
> f 2;;  
val it : int = 9
```

```
> !x;;  
val it : int = 3
```

Values vs Reference Cells

```
> let x = 15;;  
val x : int = 15
```

```
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int
```

```
> f 2;;  
val it : int = 9
```

```
> x;;  
val it : int = 15
```

```
> let x = ref 15;;  
val x : int ref = { contents = 15 }
```

```
> let f y =  
    x := y + 1  
    !x * !x;;  
val f : y:int -> int
```

```
> f 2;;  
val it : int = 9
```

```
> !x;;  
val it : int = 3
```

Values vs Reference Cells

```
> let x = 15;;  
val x : int = 15
```

```
> let f y =  
    let x = y + 1  
    x * x;;  
val f : y:int -> int
```

```
> f 2;;  
val it : int = 9
```

```
> x;;  
val it : int = 15
```

```
> let x = ref 15;;  
val x : int ref = { contents = 15 }
```

```
> let f y =  
    x := y + 1  
    !x * !x;;  
val f : y:int -> int
```

```
> f 2;;  
val it : int = 9
```

```
> !x;;  
val it : int = 3
```


Values vs Reference Cells

```
> let x = 15;;
val x : int = 15
```

```
> let f y =
    let x = y + 1
    x * x;;
val f : y:int -> int
```

```
> f 2;;
val it : int = 9
```

```
> x;;
val it : int = 15
```

```
> let x = ref 15;;
val x : int ref = { contents = 15 }
```

```
> let f y =
    x := y + 1
    !x * !x;;
val f : y:int -> int
```

```
> f 2;;
val it : int = 9
```

```
> !x;;
val it : int = 3
```


Aliasing of Reference Cells

```
> let x = ref 5;;  
val x : int ref = {contents = 5}  
  
> let y = x;;  
val y : int ref = {contents = 5}  
  
> y := 10;;  
val it : unit = ()  
  
> y;;  
val it : int ref = {contents = 10}  
  
> x;;  
val it : int ref = {contents = 10}
```

- Unlike mutable variables, reference cells can be **aliased**
- That is, two variables may refer to the **same reference cell**
- Changing the reference cell through one variable can be observed via the other variable as well.

Aliasing of Reference Cells

```
> let x = ref 5;;  
val x : int ref = {contents = 5}  
  
> let y = x;;  
val y : int ref = {contents = 5}  
  
> y := 10;;  
val it : unit = ()  
  
> y;;  
val it : int ref = {contents = 10}  
  
> x;;  
val it : int ref = {contents = 10}
```

- Unlike mutable variables, reference cells can be **aliased**
- That is, two variables may refer to the **same reference cell**
- Changing the reference cell through one variable can be observed via the other variable as well.

Reference Cells vs Mutable Variables

```
> let x = ref 5;;  
val x : int ref = {contents = 5}  
  
> let y = x;;  
val y : int ref = {contents = 5}  
  
> y := 10;;  
val it : unit = ()  
  
> y;;  
val it : int ref = {contents = 10}  
  
> x;;  
val it : int ref = {contents = 10}
```

```
> let mutable x = 5;;  
val mutable x : int = 5  
  
> let mutable y = x;;  
val mutable y : int = 5  
  
> y <- 10;;  
val it : unit = ()  
  
> y;;  
val it : int = 10  
  
> x;;  
val it : int = 5
```


Reference Cells vs Mutable Variables

```
> let x = ref 5;;  
val x : int ref = {contents = 5}  
  
> let y = x;;  
val y : int ref = {contents = 5}  
  
> y := 10;;  
val it : unit = ()  
  
> y;;  
val it : int ref = {contents = 10}  
  
> x;;  
val it : int ref = {contents = 10}
```

```
> let mutable x = 5;;  
val mutable x : int = 5  
  
> let mutable y = x;;  
val mutable y : int = 5  
  
> y <- 10;;  
val it : unit = ()  
  
> y;;  
val it : int = 10  
  
> x;;  
val it : int = 5
```

Sequencing

Similarly to imperative languages we can string together expressions sequentially with `;`

`e1; e2`

- This will first evaluate `e1` and then `e2`
- This expression has the same type as `e2`
- If `e1` has any other type than `unit` then a warning will be generated
- `;` separates expressions, it does not end them

Sequencing

We have already seen sequencing.
But we used a new line instead of ;

```
let f y =  
  x := y + 1  
  !x * !x
```

Sequencing

We have already seen sequencing.
But we used a new line instead of ;

let f y =

x := y + 1

!x * !x

This is an expression of type unit

This is an expression of type int

Sequencing

We have already seen sequencing.
But we used a new line instead of ;

let f y =

x := y + 1

!x * !x

This is an expression of type unit

This is an expression of type int

The following definition is equivalent to the one above:

let f y = x := y + 1 ; !x * !x

You can do weird stuff

```
> let ( * ) x y = x := 5; !x * y ;;  
val ( * ) : x:int ref -> y:int -> int
```

```
> let x = ref 10;;  
val x : int ref = {contents = 10}
```

```
> x * 7;;  
val it : int = 35
```

```
> x;;  
val it : int ref = {contents = 5}
```

You can do weird stuff

```
> let ( * ) x y = x := 5; !x * y ;;  
val ( * ) : x:int ref -> y:int -> int
```

```
> let x = ref 10;;  
val x : int ref = {contents = 10}
```

```
> x * 7;;  
val it : int = 35
```

```
> x;;  
val it : int ref = {contents = 5}
```

There are very few places where references are useful (and **never** do this particular thing)

Mutable variables vs reference cells

Which one do we choose?

- We nearly always **choose mutable variables** over reference cells
 - Because **aliasing** is often a very bad idea!
- Reference cells are most frequently found in object-oriented code inside class declarations

Recall: Record Types

Records allow us to
structure data

```
type person =  
  { firstname : string;  
    lastname  : string;  
    age       : int }
```

- By default, records are **immutable**.
- To change a record, we have to make a new copy with changed values

Recall: Record Types

Records allow us to structure data

```
type person =  
  { firstname : string;  
    lastname  : string;  
    age       : int }
```

- By default, records are **immutable**.
- To change a record, we have to make a new copy with changed values

```
> let patrick =  
  { firstname = "Patrick";  
    lastname  = "Bahr";  
    age = 40 } ;;
```

```
> let olderPatrick =  
  { patrick with age = 41 };;
```

```
> patrick.age;;  
val it : int = 40
```

```
> olderPatrick.age;;  
val it : int = 41
```


Mutable Record Fields

We can make record fields mutable using the `mutable` keyword

```
type person =  
  { firstname    : string;  
    lastname     : string;  
    mutable age  : int }
```

```
> let patrick =  
  { firstname = "Patrick";  
    lastname  = "Bahr";  
    age = 40 } ;;  
val patrick: person = ...
```

```
> patrick.age;;  
val it : int = 40
```

```
> patrick.age <- 41;;  
val it: unit = ()
```

```
> patrick.age;;  
val it : int = 41
```

Mutable Record Fields

We can make record fields mutable using the `mutable` keyword

```
type person =  
  { firstname    : string;  
    lastname     : string;  
    mutable age  : int }
```

```
> let patrick =  
  { firstname = "Patrick";  
    lastname  = "Bahr";  
    age = 40 } ;;  
val patrick: person = ...
```

```
> patrick.age;;  
val it : int = 40
```

```
> patrick.age <- 41;;  
val it: unit = ()
```

```
> patrick.age;;  
val it : int = 41
```

Reference cells revisited

Reference cells are actually record types with a single mutable field.

```
type 'a ref =  
  { mutable contents: 'a }  
let ref v = {contents = v}
```

We could have defined `ref` ourselves like this.

Reference cells revisited

Reference cells are actually record types with a single mutable field.

```
type 'a ref =  
  { mutable contents: 'a }  
let ref v = {contents = v}
```

We could have defined `ref` ourselves like this.

```
let (!) r = r.contents  
let (:=) r x = r.contents <- x
```

Also the dereferencing operator `!` and the assignment operator `:=` can be implemented.

Quiz

Questions?

Part III

Arrays

Arrays

- Arrays in F# behave similarly to arrays in imperative languages
- Arrays are not purely functional
- They still have many uses for performance reasons

Array creation

Array literals

```
> [|1; 2; 3; 4; 5|];;  
val it : int [] = [|1; 2; 3; 4; 5|]
```

Similar to **list** literals
[1; 2; 3; 4; 5]

Array creation

```
> Array.create 5 3;;  
val it : int [] = [|3; 3; 3; 3; 3|]
```

Array initialisation

```
> Array.init 5 (fun x -> x * 2);;  
val it : int [] = [|0; 2; 4; 6; 8|]
```


Array access & update

```
> let arr = Array.create 5 3;;  
val arr : int [] = [|3; 3; 3; 3; 3|]  
  
> arr[3];;  
val it : int = 3  
  
> arr[3] <- 42;;  
val it : unit = ()  
  
> arr;;  
val it : int [] = [|3; 3; 3; 42; 3|]  
  
> arr[3];;  
val it : int = 42
```

Note that updating the array is a side effect

While-loops

While loops do exist, and do not behave like you are used to

```
while b do e
```

While b is true, do e, and throw away the result - only works on side effects

A while expression always has
type unit

An example

```
> let f arr =  
    let mutable x = 0;  
    while x < Array.length arr do  
        arr[x] <- arr[x] * 3;  
        x <- x + 1;;  
val f : arr:int [] -> unit  
  
> let arr = Array.init 5 (fun x -> x + 7);;  
val arr : int [] = [|7; 8; 9; 10; 11|]  
  
> f arr  
val it : unit = ()  
  
> arr  
val it : int [] = [|21; 24; 27; 30; 33|]
```


An example

```
> let f arr =  
    let mutable x = 0;  
    while x < Array.length arr do  
        arr[x] <- arr[x] * 3;  
        x <- x + 1;;  
val f : arr:int [] -> unit  
  
> let arr = Array.init 5 (fun x -> x + 7);;  
val arr : int [] = [|7; 8; 9; 10; 11|]  
  
> f arr  
val it : unit = ()  
  
> arr  
val it : int [] = [|21; 24; 27; 30; 33|]
```

This might seem familiar, but do not use loops.

We dock points if you do.

Array map

```
> let arr = Array.init 5 (fun x -> x + 7);;  
val arr : int [] = [|7; 8; 9; 10; 11|]  
  
> let arr3 = Array.map (( * ) 3) arr;;  
val arr3 : int [] = [|21; 24; 27; 30; 33|]  
  
> arr;;  
val arr : int [] = [|7; 8; 9; 10; 11|]
```

- Produces shorter & cleaner code, and
- Avoids risk for infinite loops.
- Note: This leaves the original array `arr` unchanged!

Iterators

Iterators are higher-order functions that use side effects

```
Array.iter : ('a -> unit) -> 'a [] -> unit)
```


Iterators

Iterators are higher-order functions that use side effects

```
Array.iter : ('a -> unit) -> 'a [] -> unit)
```

```
> Array.iter (printfn "element: %d") [|7; 8; 9; 10; 11|];;
```

```
element: 7
```

```
element: 8
```

```
element: 9
```

```
element: 10
```

```
element: 11
```

```
val it : unit = ()
```

Iterators

Iterators are higher-order functions that use side effects

```
Array.iter : (('a -> unit) -> 'a [] -> unit)
```

```
> Array.iter (printfn "element: %d") [|7; 8; 9; 10; 11|];;
```

```
element: 7  
element: 8  
element: 9  
element: 10  
element: 11  
val it : unit = ()
```

equivalently:

```
(fun x -> printfn "element: %d" x)
```


Iterators

Iterators are higher-order functions that use side effects

```
Array.iteri : ((int -> 'a -> unit) -> 'a [] -> unit)
```


Iterators

Iterators are higher-order functions that use side effects

```
Array.iteri : ((int -> 'a -> unit) -> 'a [] -> unit)
```

Iterators

Iterators are higher-order functions that use side effects

```
Array.iteri : ((int -> 'a -> unit) -> 'a [] -> unit)
```

```
> Array.iteri (printfn "index: %d\t element: %d") [|7; 8; 9; 10; 11|]
```

```
index: 0      element: 7
index: 1      element: 8
index: 2      element: 9
index: 3      element: 10
index: 4      element: 11
val it : unit = ()
```

Iterators

Iterators are higher-order functions that use side effects

```
Array.iteri : ((int -> 'a -> unit) -> 'a [] -> unit)
```

```
> Array.iteri (printfn "index: %d\t element: %d") [|7; 8; 9; 10; 11|]
```

```
index: 0      element: 7
index: 1      element: 8
index: 2      element: 9
index: 3      element: 10
index: 4      element: 11
val it : unit = ()
```

equivalently:

```
(fun i x ->
  printfn "index: %d\t element: %d" i x)
```


Array.map vs Array.iteri

```
> let arr = Array.init 5 (fun x -> x + 7);;  
val arr : int [] = [|7; 8; 9; 10; 11|]  
  
> let arr3 = Array.map (( * ) 3) arr;;  
val arr3 : int [] = [|21; 24; 27; 30; 33|]  
  
> arr;;  
val arr : int [] = [|7; 8; 9; 10; 11|]
```

Array.map
produces a
new array

```
> Array.iteri (fun i v -> arr[i] <- v * 2) arr;;  
val it: unit = ()  
  
> arr;;  
val it: int array = [|14; 16; 18; 20; 22|]
```

Here we
update the
array in place

Array

Many more functions in Array library, that mirror functions on lists:

```
mapi      : (int -> 'T -> 'U) -> 'T [] -> 'U []  
filter    : ('T -> bool) -> 'T [] -> 'T []  
fold      : ('S -> 'T -> 'S) -> 'A -> 'T [] -> 'A  
foldBack  : ('T -> 'S -> 'S) -> 'T [] -> 'A -> 'A  
forall    : ('T -> bool) -> 'T [] -> bool  
exists    : ('T -> bool) -> 'T [] -> bool
```

Questions?